40. Bellek Sızıntısı (Memory Leak) ve CPU Boğulması Teşhisi

Sunucunun Her 4 Saatte Bir Şişip Çökmesini Node Profiler ile Önlemek
Bölüm 7 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

🎙️ **NotebookLM & Audio Overview Podcast Notları**

Podcast Tartışma Sorusu:
"Sunucunuz her gün saat 16:00'da RAM'i tüketip çöküyor ve yeniden başlıyor. Bu gizli bellek sızıntısını (Memory Leak) yapay zeka ve Heap Snapshot ile nasıl teşhis ederiz?"

Bölümden Çıkarılacak 3 Altın Cümle:

- 📌 Bellek Sızıntısı (Memory Leak), artık ihtiyaç duyulmayan nesnelerin referansları silinmediği için Garbage Collector tarafından temizlenememesidir. - 📌 En yaygın sızıntı kaynakları: Temizlenmeyen Event Listener'lar, global dizilere sonsuz veri eklemek ve kapanış (Closure) tuzaklarıdır.
- 📌 Chrome DevTools / Node Inspector ile alınan iki 'Heap Snapshot' karşılaştırılarak hangi nesnelerin biriktiği saniyeler içinde bulunur.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Evinizde bir çöp kovası olduğunu düşünün.

Her gün eve alışveriş poşetleri, kargo kutuları geliyor. Siz bunları kullanıp çöpe atıyorsunuz ve haftada iki gün çöpçü gelip kovayı boşaltıyor (Garbage Collector).

Peki ya siz kargo kutularını çöpe atmak yerine: *"Aman belki lazım olur"* diyerek yatak odanızın tavanına doğru üst üste yığmaya başlarsanız ne olur? 3 ay sonra odaya sığamaz hale gelirsiniz, kapı kapanmaz ve ev çöker!

Yazılımda **Bellek Sızıntısı (Memory Leak)** tam olarak budur: Geliştirici global bir diziye her gelen isteğin logunu atar. 10 gün sonra sunucunun 8 GB RAM'i biter ve işletim sistemi uygulamayı acımasızca öldürür (OOM - Out of Memory Kill).

🧠 2. AI Çağında Yeni Zihniyet Modeli

Garbage Collection ve Referans Tutma Mantığı:

```
[Kök Nesne: Global Scope / Window / Process]
                    |
            (Aktif Referans Var)
                    ▼
          [GlobalCache Dizisi]
                    |
            (Binlerce Eski İstek)
                    ▼
          [UserSession Objesi] ──> GARBAGE COLLECTOR DİYOR Kİ:
                                   "Buna hâlâ globalden erişim var,
                                    bu yüzden bu nesneyi RAM'den SİLEMEM!"
_____________________________________________________________________
ÇÖZÜM: Referansı kopar (session = null) veya LRU Cache kullan!
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Node.js Performans ve Profiling Mimarı
Aşağıdaki Express kodunda gizli ve ölümcül bir bellek sızıntısı (Memory Leak)
bulunmaktadır.

## KOD:
```typescript
const requestHistory = []; // Global dizi

app.get("/api/data", (req, res) => {
  const context = {
    ip: req.ip,
    headers: req.headers,
    timestamp: new Date()
  };
  requestHistory.push(context); // Her istekte şişiyor!
  res.json({ message: "Başarılı" });
});
````

GÖREV:

1. Bu kodun neden OOM (Out Of Memory) çöküşüne yol açacağını açıkla.
````

2. Sınırsız büyüyen dizi yerine 'LRU (Least Recently Used) Cache' mantığını uygulayan güvenli kodu yaz.
3. Node.js'te `--inspect` ve Chrome DevTools ile Heap Snapshot alma adımlarını özetle.

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Felaket: Event Listener temizlenmiyor ve global dizi sınırsız büyüyor
const EventEmitter = require('events');
const eventBus = new EventEmitter();

app.get("/subscribe", (req, res) => {
  // Her istekte yeni bir dinleyici eklenir ama asla silinmez! (Memory Leak)
  eventBus.on("notification", (msg) => {
    res.write(msg);
  });
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
import { lru } from "tiny-lru";

// 1. Sınırlı Bellek: En fazla 1.000 kayıt tutan LRU Cache (Sabit RAM Kullanımı)
const requestCache = lru<RequestContext>(1000);

export function trackRequestSafe(req: Request) {
  const requestId = crypto.randomUUID();
  requestCache.set(requestId, {
    ip: req.ip,
    timestamp: Date.now()
  });
  // 1001. kayıt geldiğinde en eski kayıt otomatik bellekten atılır (Eviction)
}

// 2. Event Listener Temizliği (Hafıza Güvenliği)
export function attachCleanListener(req: Request, res: Response) {
  const onNotification = (data: string) => {
    res.write(data);
  };

  eventBus.on("notification", onNotification);

  // İstemci bağlantıyı kapattığında dinleyiciyi MUTLAKA kaldır!
  req.on("close", () => {
    eventBus.removeListener("notification", onNotification);
  });
}
```
```

💡 **Kod Analizi:** `lru(1000)` kullanılarak bellek tüketimi tavan bir sınırla sabitlenmiş; `req.on('close')` içinde `removeListener` çağrılarak bağlantı kapandığında nesnenin bellekten temizlenmesi garanti edilmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Geçici verileri global değişkenlerde veya dosya kapsamındaki (module-level) dizilerde saklamak.
  - **Çözüm:** Global state'ten kaçının; saklanması gereken verileri Redis veya veritabanına yazın.
- **Tuzak:** Event listener ekleyip (`socket.on`) bağlantı koptuğunda temizlemeyi (`socket.off`) unutmak.
  - **Çözüm:** Her listener için mutlaka bir cleanup (temizlik) fonksiyonu tanımlayın.
- **Tuzak:** Büyük resim veya dosya işlemlerini belleğe (Buffer) tek parça yüklemek.
  - **Çözüm:** Büyük verileri işlerken RAM'i şişirmemek için Stream (Akış) API'lerini kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📹 Ders Başlığı: *Ders 40: Bellek Sızıntısı & CPU Profiling - OOM Çöküşlerini Önlemek*

- 🖥️ **Slayt 1:** Bellek Sızıntısı (Memory Leak) nedir? V8 Garbage Collector nasıl çalışır?
- 🖥️ **Slayt 2:** En yaygın 3 sızıntı tuzağı: Global arrays, uncleaned listeners, closures.
- 🖥️ **Slayt 3:** Chrome DevTools ile Heap Snapshot alma ve karşılaştırma.
- 🖥️ **Slayt 4:** Stream kullanarak devasa dosyaları sıfır RAM ile işlemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Node uygulamasını `node --inspect index.js` ile başlatın. Chrome'da `chrome://inspect` sayfasını açın. Heap Snapshot alın. Koda kasıtlı bir sızıntı ekleyip ikinci snapshot'ı alın. 'Comparison' görünümünde hangi objenin biriktiğini canlı kanıtlayın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizi `node --inspect` ile çalıştırıp Chrome DevTools üzerinde ilk Heap Snapshot'ınızı alın ve bellekte en çok yer tutan ilk 3 nesne tipini inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Node.js (V8) Garbage Collector'ı bellekteki bir nesneyi neden silemez ve bellek sızıntısına (Memory Leak) yol açar? - A) Bilgisayarın pili bittiği için - B) Nesneye hâlâ kök (Global Scope, aktif Closure veya temizlenmemiş Listener) üzerinden ulaşan aktif bir referans zinciri bulunduğu için - C) Sadece C++ dilinde kod yazıldığı için - D) Dosya isimleri çok uzun olduğu için > **Doğru Yanıt:** B > **Açıklama:** Garbage Collector sadece hiçbir kök nesneden ulaşılamayan (unreachable) nesneleri temizler. Bir referans unutulduğunda nesne sonsuza kadar bellekte kalır.

Soru 2: 1 GB boyutundaki büyük bir CSV dosyasını sunucunun 512 MB RAM'ini patlatmadan işlemek için hangi mimari yaklaşım kullanılmalıdır? - A) Dosyayı tek seferde `fs.readFileSync` ile hafızaya almak - B) Node.js Streams (Akış) kullanarak dosyayı parça parça (chunk by chunk) okuyup işlemek - C) Dosyayı silmek - D) Tarayıcıyı kapatmak > **Doğru Yanıt:** B > **Açıklama:** Stream'ler veriyi tümüyle RAM'e almak yerine küçük tamponlar (buffer) halinde sırayla işler; böylece dosya ne kadar büyük olursa olsun RAM kullanımı sabit kalır.